



What is CSS User Interface?

CSS User Interface properties control how users interact with your webpage. They include cursor styles, element outlines, resizing behavior, and other interactive elements that provide visual feedback and improve user experience.

These properties are crucial for creating intuitive, accessible interfaces that communicate to users what actions are possible.

Browser Support: Most UI properties are well-supported across all modern browsers. Some advanced properties may require vendor prefixes in older browsers.

The cursor Property

What is cursor?

The **cursor** property specifies which mouse cursor to display when hovering over an element. This provides visual feedback about what action will occur on click.

Syntax

Syntax: cursor: <value>;

Values: Keyword values or url() to custom image

Keyword Cursor Values

```
/* General */
cursor: auto; /* Browser default */
cursor: default; /* Arrow pointer */
cursor: none; /* No cursor visible */

/* Links & Actions */
cursor: pointer; /* Hand/finger (clickable) */
cursor: help; /* Question mark */

/* Text */
cursor: text; /* I-beam (text selection) */
cursor: move; /* Four-directional arrows */
cursor: copy; /* Copy symbol */
cursor: not-allowed; /* Prohibited/disabled */

/* Resizing */
cursor: n-resize; /* Resize top */
cursor: s-resize; /* Resize bottom */
cursor: e-resize; /* Resize left */
cursor: w-resize; /* Resize right */
cursor: se-resize; /* Resize diagonal */

/* Interaction */
cursor: wait; /* Hourglass/loading */
cursor: grab; /* Open hand */
cursor: grabbing; /* Closed hand */
cursor: col-resize; /* Column resize */
cursor: row-resize; /* Row resize */
cursor: zoom-in; /* Magnifying glass + */
cursor: zoom-out; /* Magnifying glass - */
cursor: crosshair; /* Crosshair/target */
```

Custom Cursor Images

```
/* Using custom image */
cursor: url('cursor.png');

/* With fallback */
cursor: url('cursor.png'), auto;

/* With hotspot position */
cursor: url('cursor.png') 5 10, auto;
/* x=5, y=10 is the clickable point */

/* Multiple images (browser chooses) */
cursor: url('cursor.svg'), url('cursor.png'), pointer;
```

Best Practices for cursor Property

- Use **pointer** for clickable elements (buttons, links)
- Use **text** for text input areas
- Use **not-allowed** for disabled elements
- Use **wait** for loading states sparingly
- Always provide a fallback keyword after url()
- Keep custom cursor files small (< 32KB)

The outline Property

What is outline?

The **outline** property draws a line around an element, outside the border. Unlike borders, outlines don't take up space in the layout and are often used for focus states and accessibility.

Syntax

Syntax: outline: [width] [style] [color];

Individual Properties:

- outline-width: thin | medium | thick | <length>
- outline-style: none | solid | dashed | dotted | double | groove | ridge | inset | outset
- outline-color: <color> | invert
- outline-offset: <length>

Outline Examples

```
/* Basic outline */
outline: 2px solid #1e3c72;

/* Dashed outline */
outline: 2px dashed blue;

/* With offset (space between element and outline) */
outline: 2px solid #1e3c72;
outline-offset: 4px;

/* Thick outline */
outline: 5px dotted red;

/* Invert (shows on any background) */
outline: 2px solid invert;
```

Focus Outlines for Accessibility

```
/* Custom focus outline */
button:focus {
  outline: 3px solid #2a5298;
  outline-offset: 2px;
}

/* Remove default outline (not recommended) */
button:focus {
  outline: none;
  box-shadow: 0 0 3px rgba(42, 82, 152, 0.5);
  /* ALWAYS provide alternative focus indicator */
}
```

outline vs border

Property	outline	border
Takes up space	No	Yes
Rounded corners	No (always rectangular)	Yes (border-radius)
Can have offset	Yes (outline-offset)	No
Use case	Focus states, debugging	Structural styling

 **Interactive UI Demos**

Cursor Examples - Hover over each box

pointer

text

move

Clickable

Text selection

Draggable

cursor: pointer;

cursor: text;

cursor: move;

wait

help

not-allowed

Loading

Help info

Disabled

cursor: wait;

cursor: help;

cursor: not-allowed;

grab

default

Draggable item

Normal

cursor: grab;

cursor: default;

Outline Examples - Click each box

No outline

Solid

Dashed

None

Solid outline

Dashed outline

outline: none;

outline: 2px solid;

outline: 2px dashed;

Offset

With offset

outline-offset: 4px;

The resize Property

What is resize?

The **resize** property specifies whether an element can be resized by the user. This is primarily used with **<textarea>** elements and other elements with **overflow: auto**.

Syntax

Syntax: resize: none | both | vertical | horizontal | block | inline;

Values:

- none (default) - Element cannot be resized
- both - Resizable in both directions
- vertical - Resizable vertically only
- horizontal - Resizable horizontally only
- block - Resizable in block direction
- inline - Resizable in inline direction

Resize Examples

```
/* Allow any resizing */
textarea {
  resize: both;
}

/* Vertical only (height) */
textarea {
  resize: vertical;
}

/* Horizontal only (width) */
textarea {
  resize: horizontal;
}

/* Disable resizing */
textarea {
  resize: none;
}

/* With min/max constraints */
textarea {
  resize: both;
  min-width: 200px;
  max-width: 500px;
  min-height: 100px;
  max-height: 300px;
}
```

Interactive Resize Demo

Try dragging the corner of each textarea to resize it:

Resizable (both directions)

Drag to resize in both directions...

Vertical only

Drag to resize vertically...

Horizontal only

Drag to resize horizontally...

Non-resizable

This cannot be resized...

The user-select Property

What is user-select?

The **user-select** property controls whether text can be selected by the user. This is useful for UI elements that shouldn't be selectable.

Syntax

Syntax: user-select: auto | none | text | all | contain;

Values:

- auto (default) - Browser default behavior
- none - Text cannot be selected
- text - Text can be selected
- all - All text selected in one click
- contain - Selection contained within element

Examples

```
/* Button text unselectable */
button { user-select: none; }

/* Code block selectable */
pre, code { user-select: text; }

/* Allow selection */
p { user-select: auto; }

/* Select all in one click */
.highlight { user-select: all; }
```

The pointer-events Property

What is pointer-events?

The **pointer-events** property specifies whether an element can be a target of pointer events (mouse clicks, touches).

Syntax

Syntax: pointer-events: auto | none | ... (many SVG values);

Common Values:

- auto (default) - Element is a target of pointer events
- none - Element is not a target of pointer events

Examples

```
/* Disabled overlay - clicks pass through */
.overlay {
  pointer-events: none;
}

/* Clickable element */
button {
  pointer-events: auto;
}

/* Disabled link - not clickable */
a.disabled {
  pointer-events: none;
  opacity: 0.5;
  cursor: not-allowed;
}
```

The caret-color Property

What is caret-color?

The **caret-color** property specifies the color of the text cursor (caret) in input fields and textareas.

Syntax

Syntax: caret-color: auto | <color>;

Values:

- auto (default) - Browser default color
- <color> - Any color value

Examples

```
/* Blue caret */
input {
  caret-color: blue;
}

/* Accent color caret */
textarea {
  caret-color: #2a5298;
}

/* Match text color */
input[type="text"] {
  color: #333;
  caret-color: #333;
}
```

Common Use Cases

Accessible Button with Custom Focus

```
button {
  cursor: pointer;
  user-select: none;
}

button:focus {
  outline: 3px solid #2a5298;
  outline-offset: 2px;
}
```

Disabled Form Element

```
input:disabled,
textarea:disabled {
  cursor: not-allowed;
  opacity: 0.6;
  background: #f0f0f0;
}
```

Draggable Element

```
[draggable="true"] {
  cursor: grab;
  user-select: none;
}

[draggable="true"]:active {
  cursor: grabbing;
}
```

Loading State

```
.loading {
  cursor: wait;
  pointer-events: none;
}
```

Styled Input with Custom Caret

```
input[type="text"] {
  border: 2px solid #2a5298;
  padding: 8px;
  caret-color: #2a5298;
  font-size: 16px;
}

input[type="text"]:focus {
  outline: none;
  box-shadow: 0 0 3px rgba(42, 82, 152, 0.1);
}
```

Best Practices

- Always indicate focus state:** Use outline or box-shadow for keyboard navigation. Never remove without replacement.
- Use semantic cursors:** **pointer** for clickable items, **text** for input fields.
- Disable selection wisely:** Only use **user-select: none** when necessary (buttons, UI elements).
- Custom cursors should be clear:** Users should immediately understand what action is possible.
- Test with keyboard:** Ensure focus states are visible with Tab key navigation.
- Don't overuse outlines:** Too many outlines create visual clutter.
- Consider accessibility:** Sufficient color contrast and clear focus indicators benefit all users.
- Use outline-offset strategically:** Positive offsets can make focus indicators stand out more.

Browser Compatibility

cursor: Excellent support. Most keywords work in all browsers.

outline: Full support in all modern browsers. outline-offset may need testing in older browsers.

resize: Well-supported. Works in all modern browsers.

user-select: Widely supported. Use **-webkit-user-select** and **-moz-user-select** prefixes for older versions.

caret-color: Modern browsers support it. Older browsers fall back to default.

Quick Reference

Property	Purpose	Common Values
cursor	Mouse cursor appearance	pointer, text, move, wait, not-allowed, grab
outline	Element border outline	2px solid #color, dashed, dotted
resize	Resizable element	both, vertical, horizontal, none
user-select	Text selection control	none, text, all, auto
pointer-events	Pointer event target	auto, none
caret-color	Text input cursor color	#color, auto